



QwenStudio

Running an open image model entirely on your own machine:
what it takes, what it gives you, and where it is worse.

This is an exploration, not a product.

Not an official Superside tool, not affiliated with or endorsed by Superside,
and not endorsed by the Qwen team or Alibaba. Qwen-Image 2.1 is published under
the Qwen Research License: personal and research use only.



What this is

An image generation and editing app that runs Qwen-Image 2.1 on one machine, with nothing in between. No ComfyUI, no cloud, no account, no API key. You clone it, double-click an installer, and 31 GB of weights come down once.

The installer downloads its own Python. It touches no system Python, installs nothing globally, and changes no PATH. Delete the folder and every trace is gone. Before installing anything it reads the hardware and picks a profile from it — which PyTorch wheel, which dtype, whether to quantise, how much to offload, what resolution to cap at — so a 10 GB card and a 48 GB card both get something that runs.

Two ways to use it

By hand. A page opens at 127.0.0.1:7860. Eight use cases, each showing only the inputs it needs, each opening with a worked example already in its boxes and the result of that exact example beside it. The first screen shows this + this = this instead of an empty form.

By agent. Point Claude Code, Codex or any MCP client at the folder. Ten tools — generate, edit, describe, batch, pose library, prompt library — drive the same engine the interface drives. The model loads once and both share it, so the page can stay open while an agent works through a list of prompts.

That second door is the point. Everything the interface can do is an HTTP endpoint; the agent layer is thin on purpose. This is not a wrapper around a script, it is a local service with two front doors.

What it does

Group	Use case	What goes in
Text to image	New image	nothing but the prompt
	Text in the image	words in quotes come out as lettering
From a photo	New portrait	person photos
	Character in a scene	person + a photo of the setting
	Pose my character	person + pose + optional style and setting
	Transparent cutout	person → PNG with a real alpha channel
	Free	everything, nothing assumed
Edit a photo	Apply a look	image + a treatment picked from a grid
	Match a style	image + a picture whose manner you want
	Rescale to 2K	image, redrawn larger rather than stretched
	Replace something	image + a selection + what goes there



An edit selects its region from words or from a brush painted at the photo's real resolution. What you leave untouched comes back byte for byte — the test suite checks that rather than assuming it.

How this differs from a closed model

Not “better”. Different in ways that decide whether it fits what you are doing.

	QwenStudio, locally	A hosted closed model
Where the image goes	nowhere — no upload, no retention, no terms about your inputs	to a server, under whatever the terms say today
Cost per image	electricity	per call, forever
Works offline	yes, once the weights are down	no
The prompt you sent	exactly what you wrote, plus scaffolding you can read in a file	rewritten by a layer you cannot see
Refusals	none of its own	a moderation policy that changes without notice
Reproducibility	same seed, same weights, same image, in a year	the model is swapped underneath you
Identity from a photo	reference images, no training run	usually not offered at all
Speed	62 s at 1 MP, 239 s at 2K, on a 24 GB laptop GPU	seconds
Peak quality	very good	usually better
Licence to sell the output	no — Qwen Research License	usually yes

The honest summary: a closed model is faster, often prettier, and legally simpler to sell. This one is private, free to run, reproducible, inspectable and yours — and the prompt scaffolding that makes identity transfer work is written in a file you can open and argue with.

That last point carries the most weight. Every rule this app applies was measured rather than assumed, and the ones that turned out wrong are still written down. A closed model hands you a result. This hands you a result and the reason.



How it works

Four reference slots, one job each

The model accepts several reference images. Left to itself it blends them; the app gives each one a job and says so in the prompt, so they compose instead of fighting.

Slot	Gives	Stays out of it
Person	the face and identity	everything else
Pose	posture, limbs, head tilt	build and height — those stay the person's
Style	grade, contrast, grain, quality of light	its subject, setting and composition
Scene	place, wardrobe, framing, lighting	the face

Each reference is tagged `<image1>`, `<image2>`... and the tag is repeated inside its own instruction, because the text encoder reserves one vision slot per tag and that binding is what keeps them apart. The list handed to the pipeline and the numbering in the prompt are built in the same function, so they cannot drift apart, and every response names which slot got which tag.

Three models, one GPU

The 7B image transformer, Qwen3-VL for description and CLIPSeg for segmentation all want the same card. A small registry knows which can coexist and swaps the rest out. The image pipeline is never unmounted: with model offload its weights live in CPU memory and cost about 0.4 GB idle, so evicting it would only buy a reload. For batch work a setting keeps everything mounted and skips the swapping.

Describing an image costs no extra download, which is the neatest thing in the build: the image model's text encoder is Qwen3-VL-8B, and the checkpoint already carries the vision tower and the language head. The same 16 GB do both jobs.

Where the images go, and how they got there

Everything written lands in one folder, and the gallery reads that folder rather than a database, so the disk is the single source of truth: delete a file there and it disappears from the gallery.

The results column is not empty the first time it is opened. It carries thirty images this install produced, in two groups, each one clickable for the prompt, the seed and the settings behind it, and for a button that opens the use case it came from with that prompt already written. Five of them build on each other — a character invented from a paragraph, carried into a portrait, a scene and a cutout, and two of those images then edited — because one tool doing a day's work argues better than eight unrelated demonstrations. It steps aside as soon as there is work of the user's own to show.

Nine paths and a handful of tools do not fit on one screen, so the footer carries an index of all of them, and each entry is named the way someone would ask for it rather than the way the panel is labelled. Nobody looking to remove a background guesses that it lives under "transparent cutout", so that is what the index calls it, and clicking opens that path already set up. The same footer carries the two sentences that matter



before an image leaves the machine: this is a beta to experiment with, and the model's licence is non-commercial.

Each result carries its own recipe, written into the PNG as tEXt chunks: the exact prompt, the seed, the steps, the decoder, the LoRA, the reference order. Click any image and it opens, with the prompt ready to copy, to put back in the box, or the image itself ready to become the input of the next step. Into the file rather than a sidecar or a database, so it survives the file being moved, copied or sent to someone.

Looks, and what was left out

A treatment is picked from a grid rather than typed, and every thumbnail in it is that effect applied to this install's own reference photo, generated on this machine. A look can be confined to a painted region instead of covering the frame. Upscaling to 2K is implemented and deliberately not offered: it works, and it took 754 seconds here for one image. Twelve minutes is not a feature. The endpoint and the reasoning are written down instead.

LoRAs, and a trap worth knowing

Drop .safetensors into the loras folder and a selector appears with a weight beside it. The catch is generational: the relight LoRAs circulating publicly were trained for Qwen-Image-Edit 2509 and 2511, a different architecture, and do not transfer to Qwen-Image 2.1. None is shipped as a use case here because none has been verified against this model. Loading one aimed elsewhere is treated as a user error rather than a crash. Reference-based identity already works without any LoRA, which is why there is no training step in this project at all.

Editing without touching the rest

An edit crops around the mask with padding, regenerates that crop at full resolution, and pastes it back through a feathered edge. Detail lands where the edit is instead of being spread thin over the whole frame, and the untouched area is preserved exactly.

Both editing paths run through that same routine, so a look confined to a painted region and a replacement are one operation with two prompts. The region can be named in words — CLIPSeg finds it, SAM 2 sharpens the edge — or painted by hand at the photograph's real resolution. A replacement needs a region; a look does not, and without one it grades the whole frame.

Knowing what the card is doing

The header carries a live reading of the card: memory used against memory fitted, refreshed while work runs, with a button that drops the cached allocations without unloading the model. Before each image of a batch the app can wait for the card to come back under a temperature you set. That is not protection from damage — the firmware enforces its own limits and nothing here can override them — but a long unattended run finishes sooner if it is not being throttled the whole way, and a number on screen beats guessing.

Deleting from the gallery moves the file to a bin folder on disk rather than unlinking it. A grid of thumbnails that look alike is exactly where an irreversible click loses the good one; emptying the bin stays the user's decision.



What we measured

All of it in one long session on an RTX 5090 Laptop with 24 GB. Where a belief turned out to be wrong, the wrong version is kept.

Reference order decides everything.

Identity transfers when the person is <image1>. With the scene first, the scene dominates and the face does not change at all — and it fails silently, producing a plausible image that is simply the wrong person. The app fixes the order: person, pose, style, scene.

A scene with a person in it will hand you that person.

The identity clause sits at the start of the prompt and the scene reference, closer to the end, outranks it. With a female scene and a male reference, the result came back as the woman from the scene: right size, right light, wrong human. The fix is recency — repeat the identity after the scene clause. Same seed, same inputs, correct face.

A reference is ignored unless the prompt names what to take from it.

With a scene loaded and a neutral prompt, the scene contributed nothing. The scaffolding clause alone was not enough. The app now describes the scene with Qwen3-VL and appends that description, which is what makes the path work at all.

"Replace X" is read as "add X on top of X".

Asking for a denim jacket over the mask of a yellow sweater returned the jacket open, with the sweater underneath. The mask had room for both and the model used it. Saying the garment is closed and that nothing shows underneath is what turns a layer into a replacement.

Prompts go positive but imperative.

At the default cfg of 1 there is no negative guidance, so "ignore the background" just injects the concept. But softening the verb breaks it too: "the face is hers" does not transfer, "the subject's face must match <image1> exactly" does.

A negative prompt does not exist below cfg 1.

The pipeline accepts a negative prompt at any cfg, and at 1 it is silently inert: same seed, byte-identical image, identical time, because the second forward pass is never run. Raised to 3 with something to push against, the same prompt turned a blurred watch movement into resolved jewels and screws for 80% more time. The control is offered only above 1, because a control that looks like it works and does not is worse than no control.



Style is a description, not a picture.

Handing the model a painting as `<image2>` and asking for its style barely moves the image, and what does come across is the painting's subject: a lighthouse landed in a portrait, and a botanical watercolour replaced the sitter with its own rosemary and fig. Naming the technique in words — flat colours, no gradients, crisp geometric edges — is what works. So the app reads the reference with Qwen3-VL first, asking only how the picture is made and never what it shows, and sends that sentence with a single image.

Steps are cheap on the clock and not cheap in the picture.

At 1 MP the clock barely moves between 8 and 25 steps — 18 s to 38 s — which made steps look free. The picture disagrees: 8 is mush, 12 leaves a watch movement soft, 16 resolves its screws, and 20 and 25 add nothing. Measuring the wrong quantity is how a default ends up at 12.

int8 quantisation was counterproductive.

The hypothesis was that it would save memory. Measured: 2.8 s/step and 24.1 GB peak, against 1.26 s/step and 21.2 GB unquantised — bitsandbytes int8 casts bf16 to fp16 and back on every matmul. It is not in the profile ladder at all.

Close anything else using the GPU first.

Starting with VRAM half full makes generation crawl with no error: utilisation reads 100%, power draw stays low, nothing finishes. Half an hour went to a forgotten ComfyUI process before the app learned to warn about it.

Numbers

1 MP (1024 px), 16 steps, warm	26 s, 7.3 GB peak
2 MP (1440 px), 16 steps	51 s, 7.3 GB peak
4 MP (2048 px, 2K native), 16 steps	122 s, 7.5 GB peak
A look or an edit, 1 MP	26 s, 9.2 GB peak
Rescale to 2K	156 s, 18.2 GB peak
Model load, first call	27–35 s
Segmentation, three phrases	0.9 s
Describe an image (Qwen3-VL, 4-bit)	5–9 s, 7.1 GB
Package size, without weights	22 MB
Weights	~31 GB, downloaded once



Those figures are nf4 on both the transformer and the text encoder, which on this architecture is not the poor mode but the good one: faster than bf16, half the memory, and the only way 2K works at all. The same seed in bf16 and in nf4 gives two different pictures, so a recipe only reproduces within one precision.

Two ceilings, not one. Generating at 2K peaks at 7.5 GB; rescaling to 2K, where the picture is its own reference, peaks at 18.2. The profile carries both numbers, because quoting only the first is how a card gets promised a size it will page on.

ComfyUI is still about four times faster for the same image: it uses int8_convrot weights with kernels built for them, a better memory manager, and prefix caching of text and reference tokens worth roughly 1.7× on edits by itself. If raw speed matters more than being self-contained, use ComfyUI. This exists to be self-contained.

How it is checked

Nineteen cases run against the live app. They check the shape of what came back, not just the absence of an exception: a 1024 square returned when 16:9 was asked for is broken even though nothing raised.

Where the instruction can be read off the image — a blazer, a kitchen, a garment that is no longer yellow — Qwen3-VL looks at the result and the case fails if the model did not do what was asked. That is what caught the layered-jacket bug: the image was the right size, the mask was right, and the edit was wrong. Shape checking alone would have passed it.

Licences

Qwen-Image 2.1	Qwen Research License — non-commercial. This is the binding one.
Qwen3-VL-8B	ships inside the same checkpoint, same licence
CLIPSeg	MIT
Inter Tight	SIL Open Font License 1.1
This code	MIT — but the model it drives is not

Nothing made with this belongs in client work without a separate licence from Alibaba. The interface borrows a palette and a typeface as a visual reference; the mark is an original symbol drawn for this repository and no trademark is used. Superside is a trademark of its owner, and this is an independent exploration with no claim of association.

The macOS path is written but has never been run. Only Windows with CUDA has been verified, and the unified-memory thresholds for Apple Silicon are judgment rather than measurement. Treat them as a starting point.